

AI ASSISTED CODING

ASSIGNMENT-7.5

Name: GEETHANJALI

H.T.No: 2303A51005

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument def

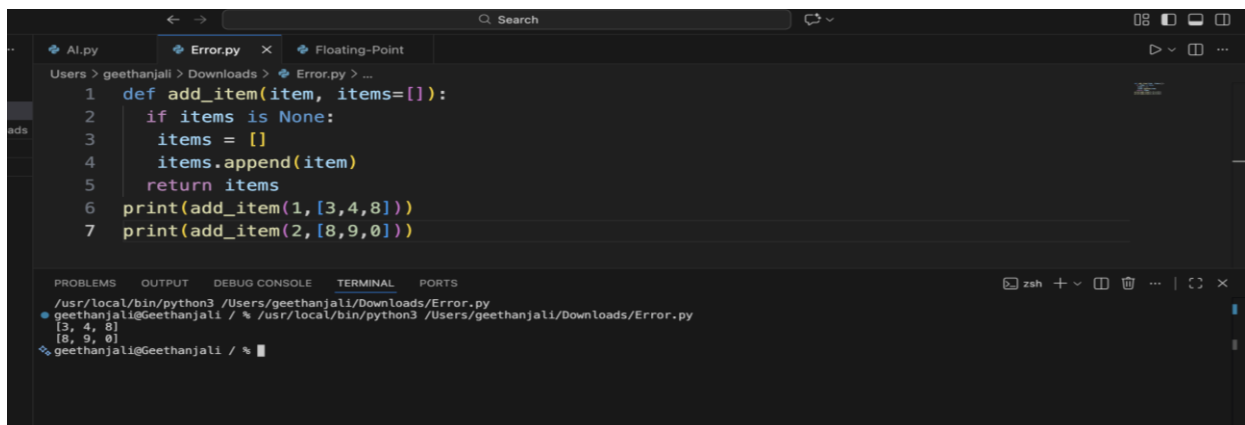
```
add_item(item, items=[]):
```

```
items.append(item) return
```

```
items print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.



The screenshot shows a code editor with a file named 'Error.py'. The code defines a function 'add_item' with a mutable default argument 'items=[]'. The function appends the item to the list and returns it. The main code calls 'add_item(1, [3,4,8])' and 'add_item(2, [8,9,0])'. The terminal output shows the results of these calls: '[3, 4, 8]' and '[8, 9, 0]'. The bug is that the first call modifies the list used in the second call, which is not the intended behavior.

```
1 def add_item(item, items=[]):
2     if items is None:
3         items = []
4         items.append(item)
5     return items
6 print(add_item(1, [3,4,8]))
7 print(add_item(2, [8,9,0]))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/Error.py
geethanjali@geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/Error.py
[3, 4, 8]
[8, 9, 0]
geethanjali@geethanjali / %
```

Task 2 (Floating-Point Precision Error)

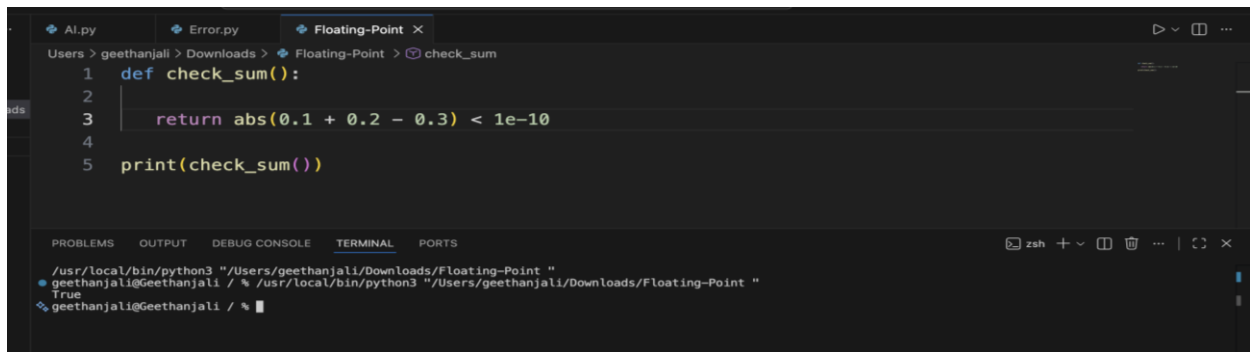
Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance.

Bug: Floating point precision issue

```
def check_sum(): return (0.1 + 0.2)
```

```
== 0.3 print(check_sum())
```

Expected Output: Corrected function



```
def check_sum():  
    return abs(0.1 + 0.2 - 0.3) < 1e-10  
print(check_sum())
```

Terminal output:
/usr/local/bin/python3 "/Users/geethanjali/Downloads/Floating-Point "
geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/Floating-Point "
True
geethanjali@Geethanjali / %

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case def

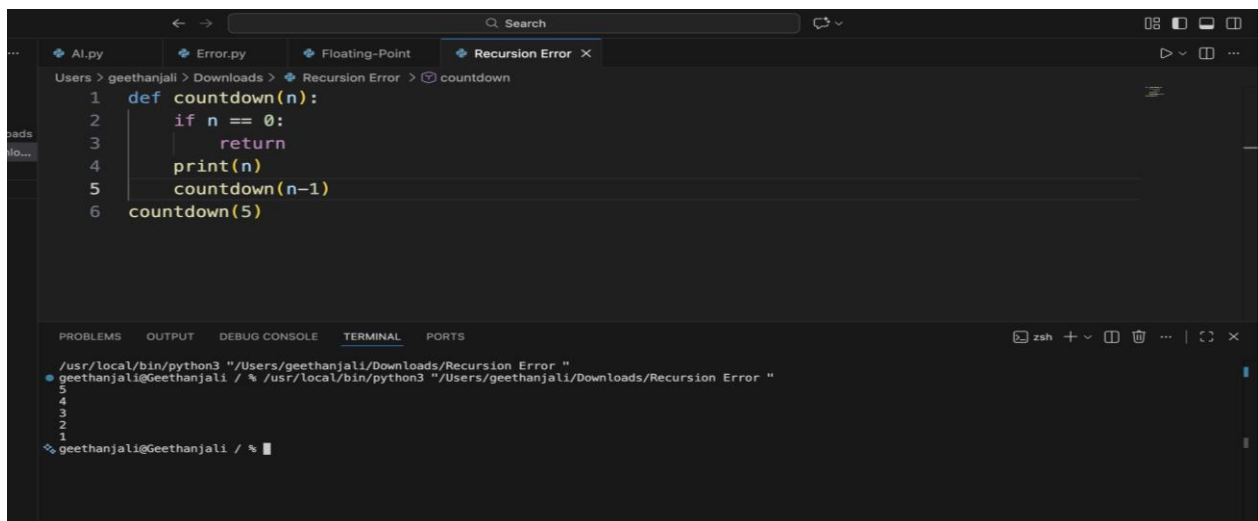
countdown(n):

print(n) return

countdown(n-1)

countdown(5)

Expected Output : Correct recursion with stopping condition.



```
def countdown(n):  
    if n == 0:  
        return  
    print(n)  
    countdown(n-1)  
countdown(5)
```

Terminal output:
/usr/local/bin/python3 "/Users/geethanjali/Downloads/Recursion Error "
geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/Recursion Error "
5
4
3
2
1
geethanjali@Geethanjali / %

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

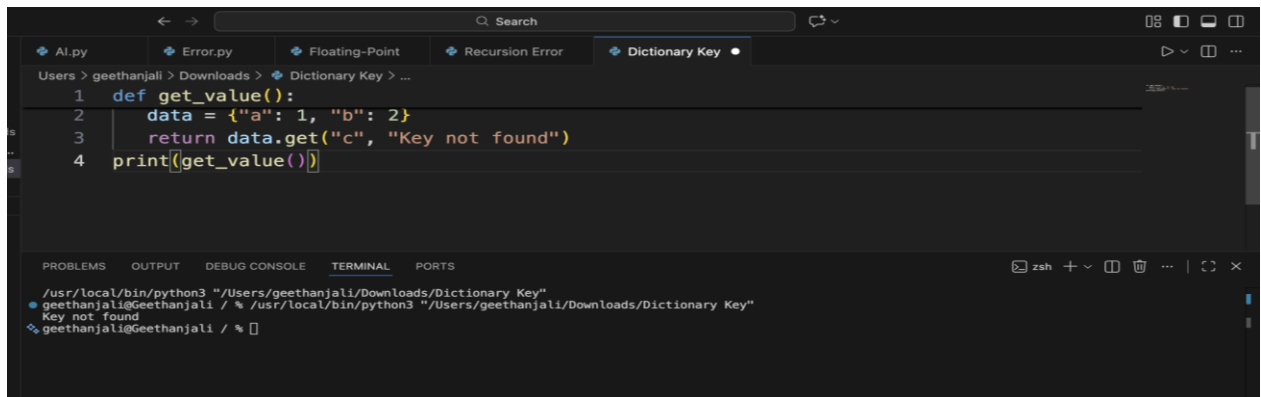
Bug: Accessing non-existing key

def get_value(): data = {"a": 1,

"b": 2} return data["c"]

print(get_value())

Expected Output: Corrected with .get() or error handling.



```
def get_value():
    data = {"a": 1, "b": 2}
    return data.get("c", "Key not found")
print(get_value())
```

Terminal Output:

```
/usr/local/bin/python3 "/Users/geethanjali/Downloads/Dictionary Key"
Key not found
```

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop def

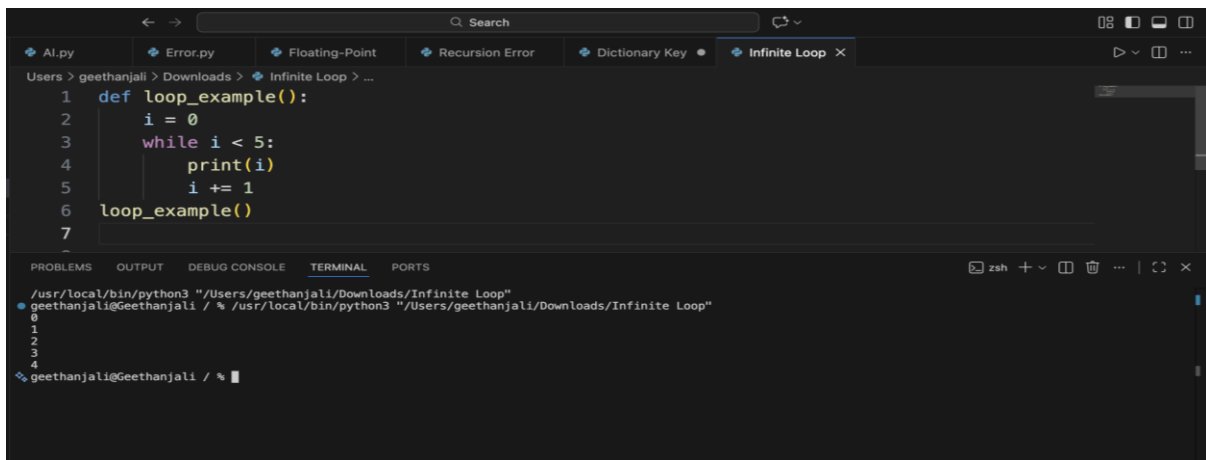
loop_example():

i = 0 while

i < 5:

print(i)

Expected Output: Corrected loop increments i.



```
def loop_example():
    i = 0
    while i < 5:
        print(i)
        i += 1
    loop_example()
```

Terminal Output:

```
0
1
2
3
4
```

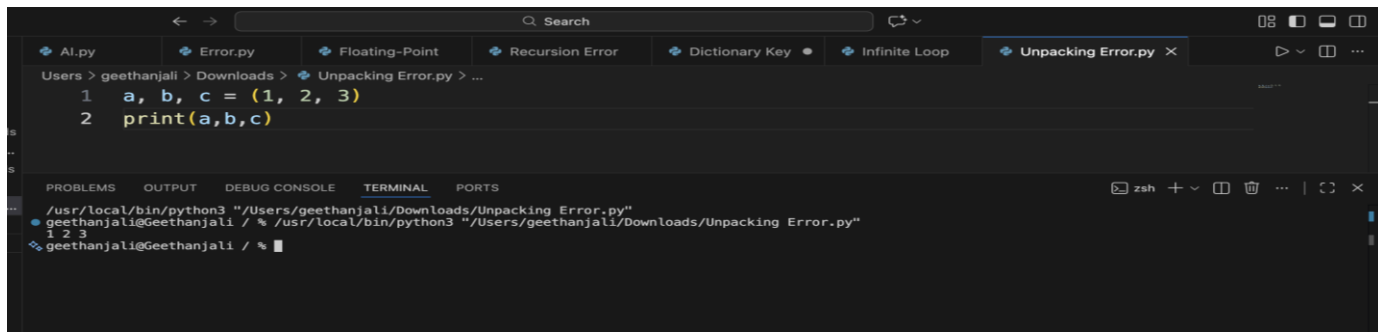
Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.



```
1 a, b, c = (1, 2, 3)
2 print(a,b,c)
```

```
/usr/local/bin/python3 "/Users/geethanjali/Downloads/Unpacking Error.py"
1 2 3
geethanjali@Geethanjali / %
```

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

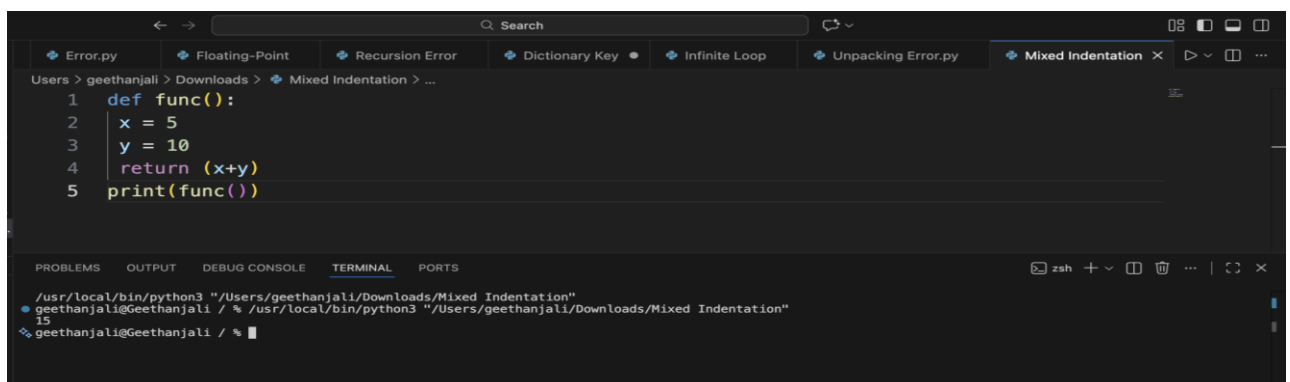
```
def func():
```

```
    x = 5 y =
```

```
    10 return
```

```
    x+y
```

Expected Output : Consistent indentation applied.



```
1 def func():
2     x = 5
3     y = 10
4     return (x+y)
5 print(func())
```

```
/usr/local/bin/python3 "/Users/geethanjali/Downloads/Mixed Indentation"
15
geethanjali@Geethanjali / %
```

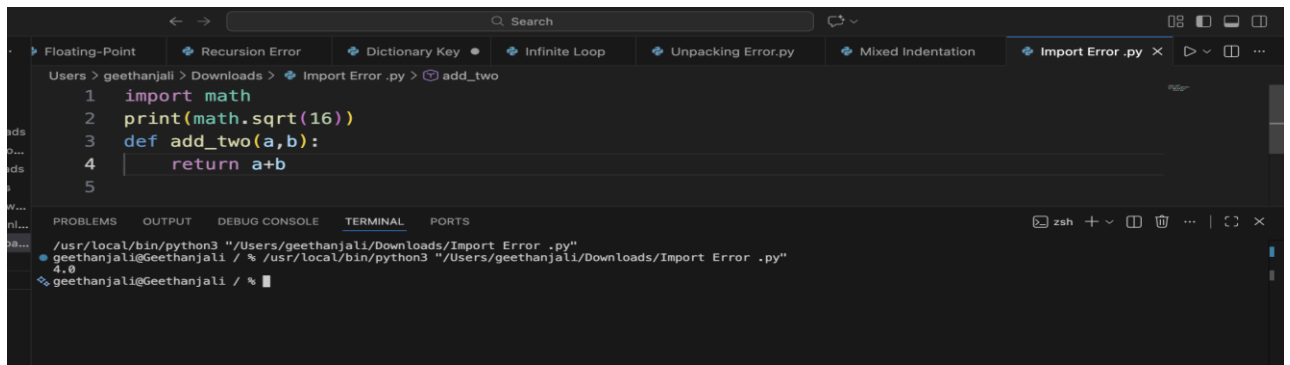
Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import import

```
maths print(maths.sqrt(16))
```

Expected Output: Corrected to import math



```
Users > geethanjali > Downloads > Import Error.py > add_two
1 import math
2 print(math.sqrt(16))
3 def add_two(a,b):
4     return a+b
5

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
/usr/local/bin/python3 "/Users/geethanjali/Downloads/Import Error.py"
geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/Import Error.py"
4.0
geethanjali@Geethanjali / %
```

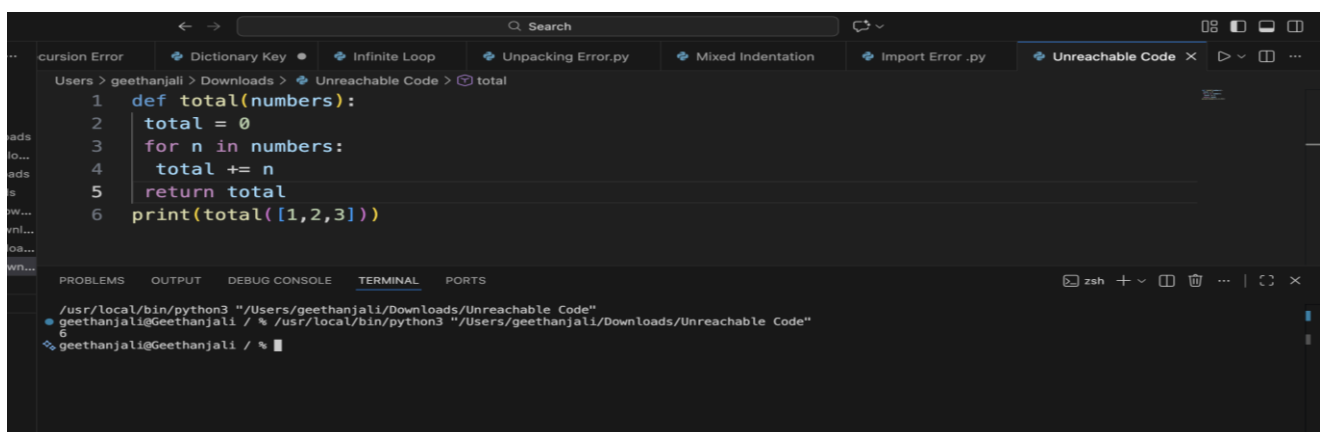
Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

```
def total(numbers):
    for n in numbers:
        return n
print(total([1,2,3]))
```

Expected Output: Corrected code accumulates sum and returns after loop.



```
Users > geethanjali > Downloads > Unreachable Code.py > total
1 def total(numbers):
2     total = 0
3     for n in numbers:
4         total += n
5     return total
6 print(total([1,2,3]))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
/usr/local/bin/python3 "/Users/geethanjali/Downloads/Unreachable Code.py"
geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/Unreachable Code.py"
6
geethanjali@Geethanjali / %
```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():
    return length * width
print(calculate_area())
```

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.

- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.

```

def calculate_area(length,width):
    return length * width
print(calculate_area(6,4))
  
```

```

/usr/local/bin/python3 "/Users/geethanjali/Downloads/Name Error"
geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/Name Error"
24
geethanjali@Geethanjali / %
  
```

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```
def add_values(): return 5 +
```

```
"10" print(add_values())
```

Requirements:

- Run the code to observe the error.
- AI should explain why `int + str` is invalid.
- Fix the code by type conversion (e.g., `int("10")` or `str(5)`).
- Verify with 3 assert cases.

Expected Output #6:

- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.

```
def add_values():  
    return 5 + int("10")  
print(add_values())
```

```
/usr/local/bin/python3 "/Users/geethanjali/Downloads/Type Error "  
geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/Type Error "  
15  
geethanjali@Geethanjali / %
```

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

```
def combine(): return
```

```
"Numbers: " + [1, 2, 3]
```

```
print(combine())
```

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

```
def combine():  
    return "Numbers: " + str([1, 2, 3])  
print(combine())
```

```
/usr/local/bin/python3 "/Users/geethanjali/Downloads/String + List"  
geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/String + List"  
Numbers: [1, 2, 3]  
geethanjali@Geethanjali / %
```

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

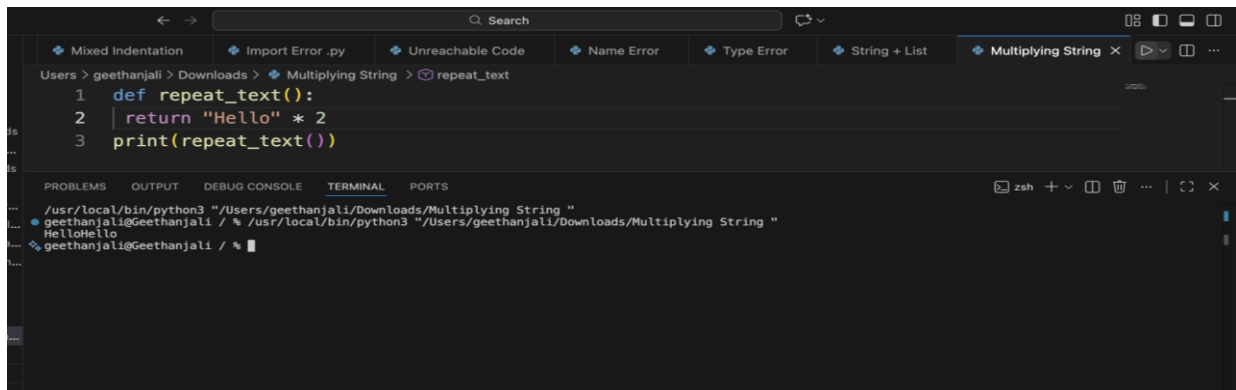
Bug: Multiplying string by float

```
def repeat_text(): return "Hello" *
```

```
2.5 print(repeat_text())
```

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.
- Add 3 assert test cases



Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

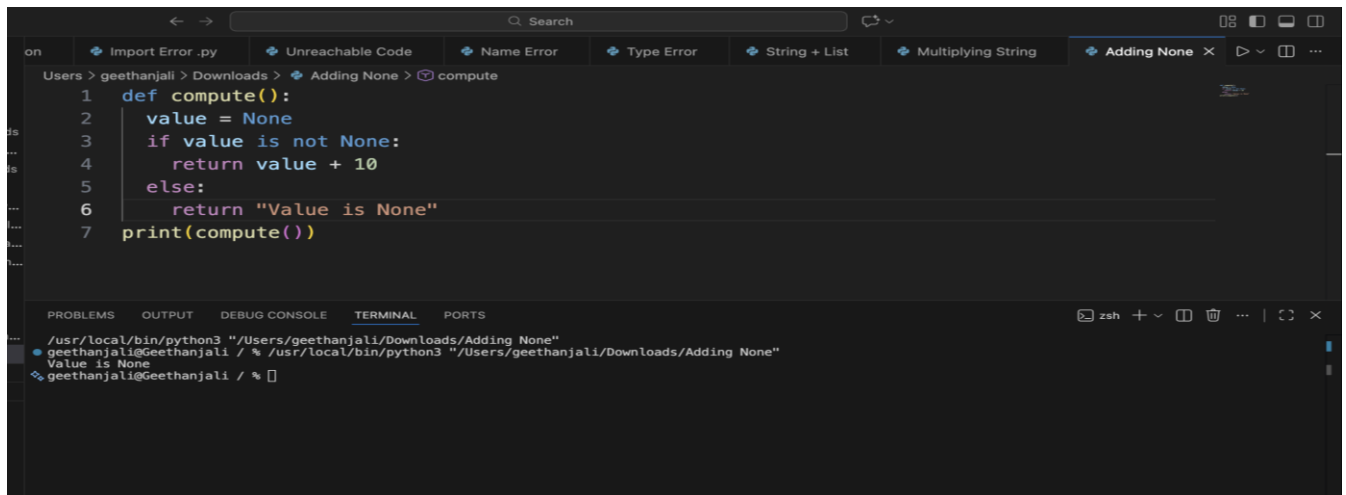
```
def compute(): value = None
```

```
return value + 10
```

```
print(compute())
```

Requirements:

- Run and identify the error.
- Explain why NoneType cannot be added.
- Fix by assigning a default value.
- Validate using asserts.



```
1 def compute():
2     value = None
3     if value is not None:
4         return value + 10
5     else:
6         return "Value is None"
7 print(compute())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 "/Users/geethanjali/Downloads/Adding None"

geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/Adding None"

Value is None

geethanjali@Geethanjali / %

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string def

sum_two_numbers():

a = input("Enter first number: ") b

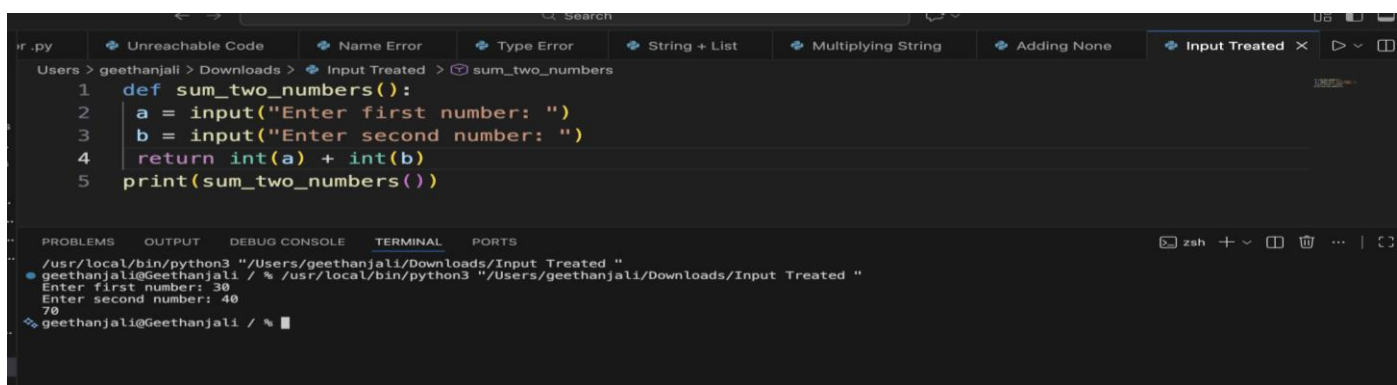
= input("Enter second number: ")

return a + b

print(sum_two_numbers())

Requirements:

- Explain why input is always string.
- Fix using int() conversion.
- Verify with assert test cases.



```
1 def sum_two_numbers():
2     a = input("Enter first number: ")
3     b = input("Enter second number: ")
4     return int(a) + int(b)
5 print(sum_two_numbers())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 "/Users/geethanjali/Downloads/Input Treated "

geethanjali@Geethanjali / % /usr/local/bin/python3 "/Users/geethanjali/Downloads/Input Treated "

Enter first number: 30

Enter second number: 40

70

geethanjali@Geethanjali / %